

Assignment: Build a simple load balancer

Overview of the Setup

The network you'll use in this exercise includes 8 hosts and a switch with an OpenFlow controller (POX). The first 4 hosts (h1 to h4) act as clients for a service offered by the hosts h5 to h8, which act as servers. The switch acts as a load balancer, and balances the flows from the clients towards the servers. The clients are addressing the public IP of the service, and the switch acts as a transparent proxy between the clients and the actual server. The setup is depicted in Fig. 1.

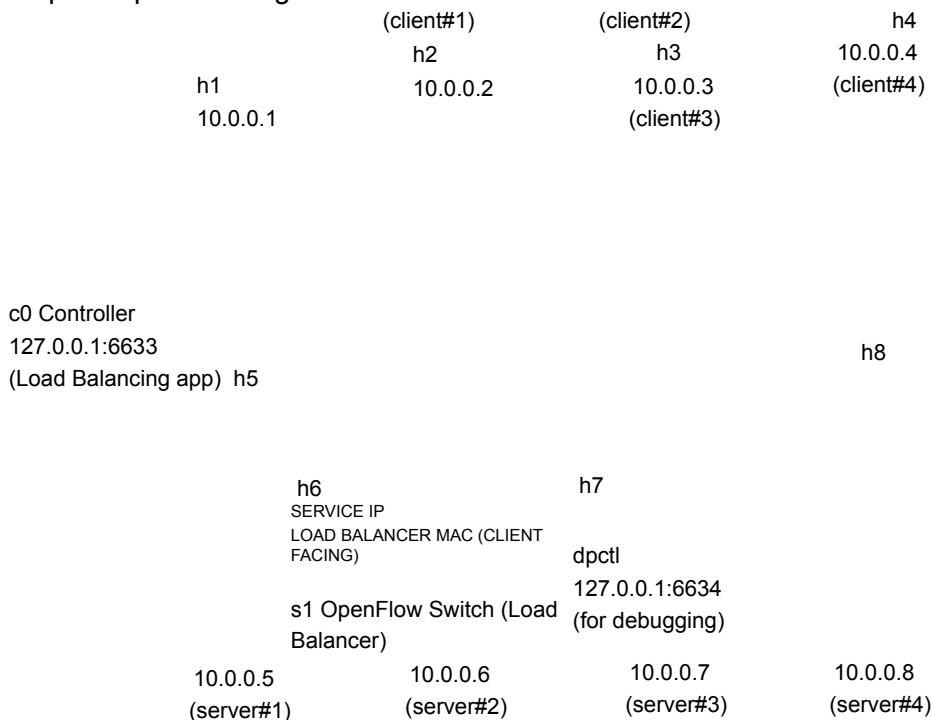


Figure 1: Load Balancer setup

What the load balancer should do

- The switch should preemptively ask for the MAC addresses of all the servers with crafted ARP requests, in order to associate these MAC addresses and the corresponding switch ports with the real IP addresses of the servers. This query should be performed upon the connection establishment of the controller to the switch in order to avoid having client flows waiting to be forwarded to the correct server. The ARP replies by the servers will be handled as part of the packet-in handler (see code skeleton later).
- Answer to ARP requests from the clients searching the MAC of the service IP address. The switch should proxy ARP replies that answer to the clients' requests with a fake MAC that is associated with the load balancer (you can use "0A:00:00:00:00:01" for simplicity). It is useful to store the information contained in each ARP request (source MAC address of client, input port of ARP request packet). In this way, when the load balancer needs later to direct flows towards the clients, it will know their MAC addresses and ports to output the packets.
- Answer to ARP requests from the servers searching the MAC addresses of clients. The switch should proxy ARP replies that answer with the fake MAC that is associated with the load balancer. At this point you should already know the MAC of the client, since it has previously requested the MAC address of the service (see previous step).

- Redirect flows from the clients towards the servers using the following load balancing mechanism: for each new IP flow from a client, select a server at random and direct the flow to this server. Of course, the server should see packets with their MAC address changed to the MAC of the load balancer, but with the source client IP intact. The destination IP address should also be rewritten to the one of the destination server. Be careful: the redirection should only happen for flows that stem from client IPs (i.e., non-server IPs) and which are directed to the service IP.
- Direct flows from the servers to the clients. This should occur after rewriting the source IP address to the one of the service and the source MAC address to the load balancer fake MAC. In this way, the clients do not see any redirection happening, and they believe that all their communication takes place between their machines and the service IP (the load balancing mechanism is transparent).
- There is no need to handle forwarding between the servers themselves or between the clients themselves; in this exercise we are interested in the load-balancing behaviour and the traffic that flows between clients and servers.

For the exercise you are requested not to use microflows (i.e. flows that match the packets exactly on all fields), but flows that match on IP addresses (while their dl type matches to the 0x800 value, i.e. the IP protocol). All the flows should expire after a 10 sec idle timeout. Default hard timeouts should not be altered. Packet-ins (corresponding to new flows) should be properly handled by your application. You should only consider ARP and IP protocol packet types.

For your convenience, a code skeleton with some initial functionality is provided below (between “CODE START” and “CODE END”):

#----- CODE START -----

```
from pox.core import core
from pox.openflow import *
import pox.openflow.libopenflow_01 as of
from pox.lib.packet.arp import arp
from pox.lib.packet.ipv4 import ipv4
from pox.lib.addresses import EthAddr, IPAddr
log = core.getLogger()
import time
import random
```

```
class SimpleLoadBalancer(object):
```

```
    def __init__(self, service_ip, server_ips = []): #initialize
        core.openflow.addListeners(self)
        #write your code here!!!
```

```
    def _handle_ConnectionUp(self, event): #new switch connection self.lb_mac =
        EthAddr("0A:00:00:00:00:01") #fake mac of load balancer self.connection =
        event.connection
        #write your code here!!!
```

```
    def update_lb_mapping(self, client_ip): #update load balancing mapping #write
        your code here!!!
```

```

def send_proxied_arp_reply(self, packet, connection, outport, requested_mac): #write your
    code here!!!

def send_proxied_arp_request(self, connection, ip):
    #write your code here!!!

def install_flow_rule_client_to_server(self, connection, outport, client_ip, server_ip,
    buffer_id=of.NO_BUFFER):
    #write your code here!!!

def install_flow_rule_server_to_client(self, connection, outport, server_ip, client_ip,
    buffer_id=of.NO_BUFFER):
    #write your code here!!!

def _handle_PacketIn(self, event):
    packet = event.parsed
    connection = event.connection
    inport = event.port
    if packet.type == packet.ARP_TYPE:
        #write your code here!!!

    elif packet.type == packet.IP_TYPE:
        #write your code here!!!

    else:
        log.info("Unknown Packet type: %s" % packet.type)
        return
    return

```

9

```

#launch application with following arguments:
#ip: public service ip, servers: ip addresses of servers (in string format) def launch(ip,
servers):
    log.info("Loading Simple Load Balancer module")
    server_ips = servers.replace(" ", "").split()
    server_ips = [IPAddr(x) for x in server_ips]
    service_ip = IPAddr(ip)
    core.registerNew(SimpleLoadBalancer, service_ip, server_ips) #-----

```

CODE END -----

The function prototypes above should be implemented by you (see “write your code here” instructions). Make sure that you log interesting messages that may help you debug your code.

Running your application

Write your application as a .py file under ~/pox/ext (name it SimpleLoadBalancer.py) Then:

```
cd ~/pox
```

Your code should run with the following arguments:

- ip: the public service ip (could be anything you want, except for the already assigned addresses 10.0.0.1-10.0.0.8)
- servers: a list of the server ip addresses (here these are the

10.0.0.5-10.0.0.8) Therefore, your application should be invoked as follows:

```
./pox.py SimpleLoadBalancer --ip=10.1.2.3  
--servers=10.0.0.5,10.0.0.6,10.0.0.7,10.0.0.8
```

Now, run Mininet with the topology described above:

```
sudo mn --topo single,8 --controller remote --mac --switch ovsk
```

Try to ping from the clients (host h1-h4) to the service IP address. Do you achieve load balancing as expected?

Please make sure that you have already studied the POX wiki [4], the Mininet documentation [1] and the OpenFlow tutorial [3] thoroughly.